

Zoho Module & Field Mapping

Which system owns which field — and who wins when they disagree

§38 DELIVERABLE

TASK P0-PMO-08

BLUEPRINT §18 · §38

VERSION 0.1

2026-09-11

STATUS	Complete for review. Two items carry an explicit OPEN marker and must be closed before the code that depends on them is written.
UNBLOCKS	P0-OPS-05 (create the custom module) · P7-BE-04 · P8-INT-01 · P8-INT-02 · P8-INT-03 · P8-INT-05 · P8-INT-07
AUTHOR	rcfworks
SUPERSEDES	Nothing. This is the first field-level mapping. §18 of the blueprint gives the object-level mapping; this document is the field-level expansion §38 requires.

CONTENTS	
1 Purpose and the boundary it enforces	8 Conflict resolution and loop prevention
2 The Zoho environment this maps onto	9 Never synced
3 Conventions used in every table below	10 Open decisions
4 Object register	11 Schema and configuration changes this document requires
5 Prerequisite Zoho configuration	12 Acceptance
6 Athlete / Content Partner — custom module build specification	13 Revision history
7 Field maps	

1 · Purpose and the boundary it enforces

§18 of the blueprint maps nine SponsorX objects onto Zoho targets and names a direction for each. It does not say which *fields* move, or which system wins when the same field is edited on both sides. Without that, the two systems drift: a sponsor's name is corrected in Zoho, corrected differently in SponsorX, and neither copy is wrong enough for anyone to notice until an invoice goes out with the wrong entity on it.

This document is the field-level answer. It exists to make one question always answerable: **for this field, which system is right?**

It restates the rule from `CLAUDE.md` that everything below obeys:

Zoho knows who we're selling to and whether they've paid. SponsorX knows what was promised, who's delivering it and whether it worked. Keep Zoho, don't build on Zoho.

Two structural consequences, both from §18 and Addendum A9:

- **Zoho never touches a request path**, in either direction. Outbound writes are worker jobs. Inbound webhooks write to our own Postgres and enqueue; they never call Zoho back.
- **Every synced object carries an external ID** so a retry cannot create a duplicate.

2 · The Zoho environment this maps onto

Read from the live org on 2026-09-11 via the CRM metadata API. **Every Zoho field API name in this document was taken from that read, not from the Zoho documentation** — so the names below are the names this org actually has.

PROPERTY	VALUE
Org name	iCARRe Foundation
Org ID (<code>zgid</code>)	749122837
Type	production (no sandbox yet — see <code>P0-OPS-04</code>)
Edition	Zoho One Enterprise, 7 user licences
Currency	USD (<code>\$</code>), matches SponsorX throughout
Timezone	PST
Primary admin	rcarr@icarrefound.org

Three environment facts that change how the sync must be written:

1. **The org timezone is PST.** `.claude/stack-decision.md` already flags that this contradicts a DMV-market business. Until it is changed, every `datetime` written to Zoho must carry an explicit UTC offset, and every `datetime` read back must be normalised to UTC before it is stored. Never send a naive timestamp. Zoho `date` fields (no time component) must be sent as a plain `YYYY-MM-DD` string derived in the *business* timezone, not by truncating a UTC instant — otherwise a due date lands a day early for a third of the year.
2. **This is the production org and there is no sandbox.** `P0-OPS-04` must deliver one before `P8-INT-07`'s backfill importer is ever run in anger.
3. **The licence `paid_expiry` reads 2026-09-18.** Almost certainly a routine renewal date, but no sync milestone should be scheduled across it unverified.

2.1 · What exists in the org today

All six stock modules §18 needs are present and API-enabled: `Accounts`, `Contacts`, `Leads`, `Deals`, `Tasks`, `Campaigns`. Also present and **deliberately unused** by SponsorX: `Quotes`, `Sales_Orders`, `Purchase_Orders`, `Invoices`, `Products`, `Price_Books`, `Vendors`, `Cases`, `Solutions`.

Two findings from that read that this document has to act on:

- **No custom module exists.** The Athlete / Content Partner module of §18 is genuinely absent — `P0-OPS-05` is real work, and §6 of this document is its build specification.
- **No external-ID field exists on any module.** §18 requires external IDs to prevent duplicates, and the org has nothing to put them in. §5 specifies the custom fields that must be added before any sync code runs. **This is unassigned click-work** — see gap **G-1** in §11.

3 · Conventions used in every table below

Direction

SYMBOL	MEANING
→	SponsorX writes to Zoho. Zoho edits to this field are ignored.
←	Zoho writes to SponsorX. SponsorX never writes this field to Zoho.
↔	Both write it. The SoR column then decides conflicts.
—	Present in Zoho, deliberately not mapped. Zoho's own value, left alone.

System of record (SoR)

The SoR column names the system whose value wins **when the same field has been changed on both sides since the last successful sync**. It is not about who writes more often; it is only the tie-break.

Where a field's SoR is neither system outright, the cell reads `SponsorX → then Zoho`, meaning: SponsorX supplies the value once at record creation and never writes it again; Zoho owns it from that moment. This pattern covers most of the CRM's sales-side fields, and it is deliberate — sales staff must be able to correct a deal name or closing date without a background job reverting them ten seconds later.

Dedupe and upsert

Every synced Zoho record carries a custom field `SponsorX_ID` holding our cuid primary key, marked *unique* and *external* in the module's field settings. All outbound writes use the Zoho **upsert** API with `duplicate_check_fields=SponsorX_ID`. A retried job therefore updates the record it created the first time instead of making a second one. `SponsorX_ID` is written once and is never updated, because our primary keys never change.

Loop prevention

Per Addendum A9 and §07 of the Implementation Guide, every Zoho-touched model carries `lastSyncOrigin` (`SPONSORX` | `ZOH0`) and `lastSyncHash` (sha256 of the stably-stringified payload). Before an outbound push, the worker recomputes the hash; if `lastSyncOrigin = ZOH0` **and** the hash is unchanged, the write is an echo of Zoho's own webhook and is dropped. §8 covers the inbound half.

Money

SponsorX stores money as **integer cents**. Zoho `currency` fields are decimal. Every crossing divides or multiplies by 100 — `budget: 823400` in Postgres is `8234.00` in Zoho. The transform belongs in the mapper functions (`toZohoDeal` / `fromZohoDeal`), never inline at a call site, so there is exactly one place for the rounding rule to live.

Names

Zoho splits people into `First_Name` (40) and `Last_Name` (80, mandatory). SponsorX stores contact names in whatever shape the source gave us. Splitting is lossy and must not be guessed: where SponsorX holds a single name string, the whole string goes to `Last_Name` and `First_Name` is left empty rather than split on the first space. Correcting it is CRM work, and Zoho is SoR for contact names anyway.

Field length

Zoho enforces field lengths server-side and rejects the whole record on overflow. Every mapper truncates to the length given in the tables below. `Account_Name` at 200 and `Deal_Name` at 120 are the two that will actually bite, because both receive composed strings.

4 • Object register

The nine §18 objects, and what each resolves to here.

#	§18 SPONSORX OBJECT	§18 ZOHU TARGET	RESOLVED TO	DIRECTION	TRIGGER	DEDUPE KEY
1	Sponsor	Accounts	<code>Accounts</code>	↔	Sponsor created / updated	<code>SponsorX_ID</code> = <code>Sponsor.id</code>
2	Sponsor Contact	Contacts	<code>Contacts</code>	↔	Contact added / updated	<code>SponsorX_ID</code> = <code>SponsorContact.id</code>
3	Lead	Leads	<code>Leads</code>	→	Inquiry received	<code>SponsorX_ID</code> = <code>Inquiry.id</code>
4	Opportunity	Deals	<code>Deals</code>	↔	Brief qualified	<code>SponsorX_ID</code> = <code>brief:<CampaignBrief.id></code>
5	Campaign	Custom Module or Campaigns	The same <code>Deals</code> record as #4 — see §4.1	↔	Campaign contracted	(same Deal)
6	Athlete / Content Partner	Custom Module recommended	New custom module — §6	→	Athlete reaches <code>APPROVED</code>	<code>SponsorX_ID</code> = <code>Athlete.id</code>
7	Task	Activities / Tasks	<code>Tasks</code>	↔	Follow-up, approval, renewal	<code>SponsorX_ID</code> = <code>SyncTask.id</code>
8	Invoice / Payment Reference	Finance / CRM fields	OPEN — O-1 (§7.7)	←	Invoice issued / paid	—
9	Renewal	Deals	A second <code>Deals</code> record	→	Campaign closing	<code>SponsorX_ID</code> = <code>renewal:<Campaign.id></code>

4.1 • Why Campaign is a Deal and not the Campaigns module

§18 leaves this open ("Custom Module or Campaigns"). It resolves to **Deals**, and the CRM `Campaigns` module goes unused.

The reason is what the stock module actually is in this org. Its `Type` picklist reads *Conference, Webinar, Trade Show, Public Relations, Partners, Referral Program, Advertisement, Banner Ads, Direct mail, Email, Telemarketing, Others, Zoho Survey, Zoho Campaigns*, its `Status` picklist is *Planning / Active / Inactive / Complete*, and it carries

nine custom fields installed by the Zoho Campaigns and Zoho Survey extensions (`Native__Campaigns__Extn__Sender_Address` and friends). It is an email-marketing object, wired to other Zoho products. A SponsorX campaign — a contracted sponsorship with a budget, a sponsor, an end date and seven states — is not that thing, and forcing it in would mean fighting two extensions for the record.

A Deal already is the right shape: it has `Amount`, `Closing_Date`, `Account_Name`, `Contact_Name`, and a `Stage` pipeline. And the sponsorship lifecycle is genuinely one commercial cycle, not two objects: a brief is the opportunity, and the campaign is that same opportunity won. §18's rows 4 and 5 are two stages of one Deal, which is why they share a record.

Consequence for the dedupe key. The Deal is created when the brief qualifies, before a `Campaign` row exists, so the key must be the brief's id. Campaign is 1:1 with brief (`Campaign.briefId` is `@unique`), so the key stays stable across the transition. For a campaign created with no brief (`Campaign.briefId` is nullable), the key is `campaign:<Campaign.id>`. A renewal is a genuinely new commercial cycle and so gets its own Deal, keyed `renewal:<Campaign.id>`.

This needs sign-off — recorded as **O-2** in §10.

5 · Prerequisite Zoho configuration

All of this is click-work in Setup → Modules and Fields, and **all of it must exist before any sync code runs**. Adding a custom field to a stock module is about a minute's work each.

5.1 · The external-ID field, on five stock modules

Add to `Accounts`, `Contacts`, `Leads`, `Deals` and `Tasks`:

PROPERTY	VALUE
Field label	<code>SponsorX ID</code>
API name	<code>SponsorX_ID</code>
Type	Single Line
Max length	50
Unique	Yes
External	Yes (this is what makes it usable as an upsert key)
Encrypted	No
On the layout	Yes, but read-only for the Standard profile — staff should see it, never edit it

5.2 · Sync-marker fields

`lastSyncOrigin` and `lastSyncHash` live **only** in Postgres. They are SponsorX's own bookkeeping and must not be created in Zoho — they would be noise on the layout and a temptation to edit.

5.3 · What must NOT be configured

- **No Zoho workflow rule may write to a field whose SoR is SponsorX.** A Zoho workflow that "helpfully" normalises `Account_Name` will fight the sync forever, and because it fires server-side the echo test cannot see it coming. Before go-live, audit the org's workflow rules against the SoR columns here.
- **No blueprint on `Deals.Stage`** that blocks the transitions §7.4 requires SponsorX to assert.

6 · Athlete / Content Partner — custom module build specification

This section is the input `P0-OPS-05` needs. Build exactly this.

6.1 · Module

PROPERTY	VALUE
Singular label	<code>Content Partner</code>
Plural label	<code>Content Partners</code>
Expected API name	<code>Content_Partners</code> — verify the generated name after creation and record it , as <code>P0-OPS-05</code> 's acceptance requires. Zoho derives the API name from the label, and a label containing <code>/</code> produces an awkward one, which is why the label drops the slash that §18's prose uses.
Description	<code>Athlete / Content Partner — SponsorX network members with a CRM-visible business relationship. Read-only mirror; SponsorX is system of record.</code>
Direction	<code>→</code> only. Nothing in this module is ever read back into SponsorX.

6.2 · Fields

Eleven fields. Every one maps to a real column in the V2 schema.

#	FIELD LABEL	API NAME	TYPE	REQ	SOURCE (<code>ATHLETE.*</code>)	NOTES
1	Content Partner Name	<code>Name</code>	Single Line (display field)	Yes	<code>displayName</code>	The module's display field. Not <code>LegalName</code> — see §6.4.
2	SponsorX ID	<code>SponsorX_ID</code>	Single Line, unique, external	Yes	<code>id</code>	Upsert key. Written once.
3	Public Profile URL	<code>Public_Profile_URL</code>	URL	No	<code>slug</code>	Composed as <code>https://<app-host>/athletes/<slug></code> . Already public.
4	Sport	<code>Sport</code>	Picklist	Yes	<code>sport</code>	Seed the picklist from the distinct values SponsorX holds at build time, and keep the two in step — a value SponsorX sends that the picklist lacks is rejected by Zoho and fails the whole record.
5	Home State	<code>Home_State</code>	Picklist (US states)	No	<code>state</code>	State only. Not <code>city</code> . See §6.4.
6	Tier	<code>Tier</code>	Picklist	No	<code>tier</code>	Values: <code>Emerging</code> , <code>Creator</code> , <code>Premium</code> , <code>Anchor</code> (§6 of the blueprint).
7	Network Status	<code>Network_Status</code>	Picklist	Yes	<code>state_</code>	Values: <code>Approved</code> , <code>Active</code> , <code>Suspended</code> — only these three . See §6.3.

#	FIELD LABEL	API NAME	TYPE	REQ	SOURCE (ATHLETE.*)	NOTES
8	Approved On	Approved_On	Date	No	first APPROVED transition	Date only, derived in the business timezone.
9	Guardian Authorisation Required	Guardian_Auth_Required	Checkbox	No	derived: birthDate implies under 18	A boolean, never the date of birth . Tells a CRM user the handling rule without exposing the minor's data.
10	Property	Property_Name	Single Line	No	property.name	Plain text, deliberately not a lookup — properties are schools, teams and venues, not CRM Accounts, and creating Accounts for them would pollute the sponsor pipeline.
11	Last Synced	Last_Synced	Date/Time	No	worker clock	Written on every successful push. Makes a stalled sync visible in the CRM itself.

6.3 · Which athlete states are ever sent

AthleteState has eight values. Only three ever reach Zoho, because §18's trigger is an *"approved network member or business relationship requiring CRM visibility"* — an applicant is not that.

ATHLETESTATE	SENT TO ZOHO AS	
DRAFT , SUBMITTED , UNDER_REVIEW , CHANGES_REQUESTED	—	No record is created at all. An application in review is not a business relationship, and pushing one would put an unvetted minor's details into the CRM.
REJECTED	—	No record. If a record already exists from a prior approval, it is set to Suspended , never deleted — deletion would destroy the CRM's history of the relationship.
APPROVED	Approved	First push happens here. This is the trigger.
ACTIVE	Active	
SUSPENDED	Suspended	

6.4 · What is deliberately excluded, and why

The user's instruction for this revision was a conservative field set, with room to expand later. That direction is also the safe one, and the asymmetry is worth stating plainly: **adding a field to a Zoho module later is a click, whereas un-syncing personal data that has already landed in the CRM is not.** Zoho retains record history, extensions have read it, and users have exported it to spreadsheets. Start narrow; widen with a decision.

Excluded from the module, all of them present in **Athlete** :

EXCLUDED	WHY
LegalName	Phase 1 moves no money through the system (P0-PM0-01 , gate G-01), so nothing in CRM needs a legal payee name. Adults-only legal name can be added later if contracting requires it.
birthDate	A minor's date of birth has no sales purpose. Field 9 carries the only fact a CRM user needs.

EXCLUDED	WHY
<code>city</code> , <code>school</code> , <code>gradYear</code>	Individually harmless; together with sport and state they identify a specific minor at a specific school. §26.
Guardian name, email, phone	Guardian records are compliance data under §4 and §11. They stay in Postgres, reachable only by roles the RBAC matrix (<code>P0-PM0-02</code>) grants.
Athlete email, phone	The athlete relationship is managed in the athlete portal, not by sales. Putting athlete contact details in CRM invites out-of-band contact that bypasses the guardian path.
<code>AthleteRate</code> , <code>Earning</code>	Rates and earnings are SponsorX's own commercial data. §18 gives Zoho the <i>sponsor</i> relationship, not athlete compensation.
<code>AthleteSocial</code> handles, followers, <code>avgViews</code>	Volatile, and provenance-labelled (§22 / <code>P0-DATA-01</code>). A number copied into Zoho arrives stripped of its <code>verified-API</code> / <code>self-reported</code> label, which is exactly the credibility failure the provenance taxonomy exists to prevent.
<code>AthleteScore</code>	Internal scoring. Phase 3 replaces the method entirely; a mirrored score would be stale and arguable.

6.5 · The expansion path

To widen this later: add the field to the module, add a row to §6.2, add a row to §6.4 explaining what changed, bump this document's version, and — for anything touching a minor — confirm against §26 first. The mapper is a single function; the cost of adding a field is one line in it.

7 · Field maps

7.1 · Sponsor ↔ Accounts

`Sponsor` is a thin model (`id`, `tenantId`, `name`). Most of what an Account holds is sales data that SponsorX has no opinion about, and the mapping says so explicitly rather than leaving it ambiguous.

SPONSORX	ZOHO (<code>ACCOUNTS</code>)	TYPE (LEN)	DIR	SOR	NOTES
<code>Sponsor.id</code>	<code>SponsorX_ID</code>	Single Line (50)	→	SponsorX	Upsert key. Written once, never updated.
<code>Sponsor.name</code>	<code>Account_Name</code>	text (200) req	↔	Zoho	Truncate at 200. Zoho wins: sales corrects the legal entity name, and the invoice has to match it.
<code>Sponsor.zohoAccountId</code>	<code>id</code>	bigint	←	Zoho	Zoho's record id, stored after the first successful upsert.
—	<code>Account_Type</code>	picklist	→ on create only	SponsorX → then Zoho	Set to <code>Customer</code> at creation. Never written again.

SPONSORX	ZOHO (ACCOUNTS)	TYPE (LEN)	DIR	SOR	NOTES
—	Phone , Website , Billing_* , Shipping_* , Industry , Ownership , Employees , Annual_Revenue , Rating , Description , Owner , Parent_Account , Account_Number , Ticker_Symbol , SIC_Code , Fax , Account_Site , Record_Image , Tag	—	—	Zoho	Zoho-only, not mirrored. SponsorX has no column for any of these and must not invent one. If a sponsor-facing screen ever needs a sponsor's address, it is a new field in Postgres with a named retrieval path, not a read-through to Zoho — Zoho never touches a request path.
Sponsor.lastSyncOrigin , .lastSyncHash	—	—	—	SponsorX	Local bookkeeping. Never sent.
Sponsor.tenantId	—	—	—	SponsorX	Never sent. Multi-tenancy is ours; Phase 1 runs one BTG tenant and the CRM has no concept of it.

7.2 · Sponsor Contact ↔ **Contacts**

Schema gap S-1. The V2 schema has no **SponsorContact** model, yet §18 requires this object bi-directionally and P4-BE-01 says "sponsor and contact records". The mapping below is written against the model that must be added — see §11.

Required model:

```

model SponsorContact {
  id          String      @id @default(cuid())
  tenantId    String
  sponsorId   String
  firstName   String?
  lastName    String      // Zoho's mandatory field
  email       String
  phone       String?
  title       String?
  isPrimary   Boolean     @default(false) // the Deal's Contact_Name
  zohoContactId String?   @unique
  lastSyncOrigin SyncOrigin?
  lastSyncHash String?
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  sponsor     Sponsor      @relation(fields: [sponsorId], references: [id])
  @@index([tenantId, sponsorId])
}

```

SPONSORX	ZOHO (CONTACTS)	TYPE (LEN)	DIR	SOR	NOTES
SponsorContact.id	SponsorX_ID	Single Line (50)	→	SponsorX	Upsert key.
.lastName	Last_Name	text (80) req	↔	Zoho	Mandatory in Zoho. Where SponsorX holds one unsplit name string, the whole string goes here.
.firstName	First_Name	text (40)	↔	Zoho	Left empty rather than guessed by splitting.
.email	Email	email (100)	↔	Zoho	Sales corrects bounced addresses in the CRM; that correction must survive.
.phone	Phone	phone (50)	↔	Zoho	
.title	Title	text (100)	↔	Zoho	
.sponsorId	Account_Name	lookup	→	SponsorX	Sent as the sponsor's zohoAccountId . If that is null the contact job blocks and re-queues behind the account push — ordering matters, and a contact with no account is CRM litter.
.zohoContactId	id	bigint	←	Zoho	

SPONSORX	ZOHO (CONTACTS)	TYPE (LEN)	DIR	SOR	NOTES
—	Email_Opt_Out , Unsubscribed_Mode , Unsubscribed_Time	boolean / picklist / datetime	←	Zoho	Consent state. Zoho is absolutely SoR. Ingest it read-only and honour it: a contact with Email_Opt_Out = true must be excluded from every SponsorX notification that is not strictly transactional. This interacts with the email-provider decision (P0-PM0-03 , gate G-04).
—	Date_of_Birth , Mailing_* , Other_* , Mobile , Home_Phone , Department , Reporting_To , Assistant , Skype_ID , Twitter , Lead_Source , Salutation , Secondary_Email , Vendor_Name , Description , Record_Image , Tag	—	—	Zoho	Zoho-only, not mirrored.
—	Parent_Name , Added_Time , Referrer_Name , Task_Owner	custom	—	Zoho	Pre-existing custom fields in this org, unrelated to SponsorX. Leave untouched.

7.3 · Lead → Leads

One-way per §18. SponsorX creates the lead; qualification happens in the CRM, which is where the sales team works.

Schema gap S-2. No model exists for an inbound inquiry. P8-INT-06 ("new enquiries become Zoho leads") has nothing to read from.

Required model:

```
model Inquiry {
  id          String    @id @default(cuid())
  tenantId    String
  companyName String?
  firstName   String?
  lastName    String
  email       String
  phone       String?
  message     String?
  source      String    // "web-form" | "outbound" | "event"
  zohoLeadId  String? @unique
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  @@index([tenantId, createdAt])
}
```

SPONSORX	ZOHO (LEADS)	TYPE (LEN)	DIR	SOR	NOTES
Inquiry.id	SponsorX_ID	Single Line (50)	→	SponsorX	Upsert key.
.lastName	Last_Name	text (80) req	→	SponsorX	
.firstName	First_Name	text (40)	→	SponsorX	
.companyName	Company	text (200)	→	SponsorX	
.email	Email	email (100)	→	SponsorX	
.phone	Phone	phone (30)	→	SponsorX	
.message	Description	textarea	→	SponsorX	
.source	Lead_Source	picklist	→ on create	SponsorX → then Zoho	Must map to an existing picklist value. web-form → OnlineStore , outbound → Cold Call , event → Trade Show . Do not send a value the picklist lacks.
.zohoLeadId	id	bigint	←	Zoho	
—	Lead_Status	picklist	—	Zoho	The entire point of pushing leads is that Zoho qualifies them. SponsorX never writes or reads this.
—	Converted__s , Converted_Account , Converted_Contact , Converted_Deal	boolean / lookup	← (proposed)	Zoho	Reading conversion back would let SponsorX know an inquiry became a sponsor. §18 marks this object one-way, so this is a scope addition, not a given — O-4 .
—	everything else	—	—	Zoho	Zoho-only.

7.4 • Opportunity / Campaign / Renewal ↔ Deals

Three §18 rows, one Zoho module. Per §4.1, the opportunity and the campaign are one Deal; a renewal is a new one.

Schema gap S-3. CampaignBrief has no zohoDealId. The Deal is created at brief qualification, before a Campaign row exists, so the brief must hold the reference. Campaign.zohoDealId already exists and is populated by copying the brief's value when the campaign is created.

SPONSORX	ZOHO (DEALS)	TYPE (LEN)	DIR	SOR	NOTES
brief:<CampaignBrief.id>	SponsorX_ID	Single Line (50)	→	SponsorX	Prefixed key — §4.1. campaign:<id> when there is no brief; renewal:<id> for a renewal Deal.

SPONSORX	ZOHO (DEALS)	TYPE (LEN)	DIR	SOR	NOTES
composed	Deal_Name	text (120) req	↔	Zoho	Composed once as <code><Sponsor.name> - <package or objective> - <Mon YYYY></code> , truncated to 120. Zoho owns it afterwards: renaming a deal is routine sales work and must not be reverted.
<code>CampaignBrief.budget</code> → <code>Campaign.budget</code>	Amount	currency (16)	↔	Split — see below	Cents ÷ 100.
<code>CampaignBrief.state</code> / <code>Campaign.state</code>	Stage	picklist req	↔	Split — see below	Mapping table below.
<code>CampaignBrief.endDate</code>	Closing_Date	date (20)	→ on create	SponsorX → then Zoho	Date only, business timezone. Sales re-forecasts freely afterwards.
<code>Sponsor.zohoAccountId</code>	Account_Name	lookup	→	SponsorX	Blocks and re-queues if null.
primary <code>SponsorContact.zohoContactId</code>	Contact_Name	lookup	→	SponsorX	The contact with <code>isPrimary = true</code> . Omitted if none exists.
<code>CampaignBrief.objective</code>	Description	textarea	→ on create	SponsorX → then Zoho	
derived	Type	picklist	→ on create	SponsorX → then Zoho	<code>New Business</code> if the sponsor has no prior <code>Closed Won</code> Deal, else <code>Existing Business</code> .
<code>Campaign.zohoDealId</code> , <code>CampaignBrief.zohoDealId</code>	id	bigint	←	Zoho	
—	Probability, Expected_Revenue, Next_Step, Lead_Source, Campaign_Source, Owner, Sales_Cycle_Duration, Overall_Sales_Duration, Lead_Conversion_Time, Stage_Modified_Time, Tag	—	—	Zoho	Zoho-only. Pipeline management is Zoho's job. <code>Campaign_Source</code> stays empty because the CRM <code>Campaigns</code> module is unused (§4.1).

Amount — a split SoR. Before the campaign exists, `Amount` is a negotiating number and **Zoho is SoR** — sales moves it around and SponsorX must not overwrite that. From `CAMPAIGN_CREATED` onward it is a contracted value backed by Campaign Orders, and **SponsorX becomes SoR**. The switch point is the same transition that sets `Stage` to `Closed Won`.

Stage — Zoho owns the pipeline, SponsorX asserts two transitions. §18 gives Zoho the opportunity pipeline, so SponsorX does not drive a deal through qualification. But two transitions are facts about our system, not sales judgement, and SponsorX asserts them:

SPONSORX STATE	STAGE WRITTEN	WHO ASSERTS
BriefState.DRAFT	(no Deal yet)	—
BriefState.QUALIFIED	Qualification	SponsorX, on create
BriefState.APPROVED	Proposal/Price Quote	SponsorX
BriefState.CAMPAIGN_CREATED	Closed Won	SponsorX asserts. The campaign is contracted; the deal is won by definition.
BriefState.CLOSED with no campaign	Closed Lost	SponsorX asserts
CampaignState.ACTIVE , REPORTING , COMPLETED	Closed Won (no change)	— · Stage never regresses once won. Delivery progress is SponsorX's business and does not belong in a sales pipeline.
CampaignState.CANCELLED	Closed Lost	SponsorX asserts
Everything in between	Zoho's own value, untouched	Zoho

Needs Analysis, Value Proposition, Id. Decision Makers, Negotiation/Review and Closed Lost to Competition are Zoho-only stages. SponsorX never writes them and must never treat an unrecognised stage as an error — a sales rep moving a deal to Negotiation/Review is normal.

Renewal Deals (§18 row 9) are → only, created when a campaign enters COMPLETED, keyed renewal: <Campaign.id>, opened at Qualification with Amount seeded from the completed campaign's budget. SponsorX never reads them back; whether the renewal closes is Zoho's business.

7.5 · Task ↔ Tasks

Schema gap S-4. No task model exists.

§18 wants follow-ups, approvals and renewals visible in the CRM. The narrow version: SponsorX pushes only tasks **it originates**, and reads back only whether they were completed. Tasks BTG staff create in Zoho stay in Zoho and never come to us.

```

model SyncTask {
  id          String      @id @default(cuid())
  tenantId    String
  subject      String
  body        String?
  dueDate     DateTime
  completedAt  DateTime?
  assigneeUserId String?   // → User.zohoUserId
  relatedDealId String?    // Zoho Deal id → What_Id
  relatedContactId String? // Zoho Contact id → Who_Id
  zohoTaskId   String?     @unique
  lastSyncOrigin SyncOrigin?
  lastSyncHash String?
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt
  @@index([tenantId, dueDate])
}

```

SPONSORX	ZOHO (TASKS)	TYPE (LEN)	DIR	SOR	NOTES
SyncTask.id	SponsorX_ID	Single Line (50)	→	SponsorX	Upsert key.
.subject	Subject	text (255) req	→	SponsorX	The field offers a picklist of suggestions (Email , Call , Meeting , ...) but accepts free text. Send our own subject.
.dueDate	Due_Date	date (20)	→	SponsorX	Date only, business timezone.
.body	Description	textarea	→	SponsorX	
.completedAt	Status	picklist	↔	Zoho	Values: Not Started , Deferred , In Progress , Completed , Waiting on someone else (note the <i>actual</i> value differs from the displayed "Waiting for input" — send the actual). SponsorX creates as Not Started ; staff work the task in the CRM, so Zoho is SoR . Inbound Completed sets completedAt .
.assigneeUserId	Owner	ownerlookup	→	SponsorX	Requires a Zoho user id — schema gap S-5: User needs zohoUserId . If unmapped, the task is created owned by the API service account, which is acceptable but means nobody is notified.
.relatedDealId	What_Id	lookup	→	SponsorX	Polymorphic in Zoho (se_module); we only ever point it at a Deal.
.relatedContactId	Who_Id	lookup	→	SponsorX	

SPONSORX	ZOHO (TASKS)	TYPE (LEN)	DIR	SOR	NOTES
—	Priority	picklist	→ on create	SponsorX → then Zoho	Always Normal on create (values: High , Highest , Low , Lowest , Normal). Staff re-prioritise freely.
—	Send_Notification_Email	boolean	→	SponsorX	Always false . SponsorX sends its own notifications; letting Zoho also email would double-notify, and the copy would be outside our control.
—	Remind_At , Recurring_Activity , Closed_Time , Tag , Last_Activity_Time	—	—	Zoho	Zoho-only.

7.6 · Athlete / Content Partner → custom module

Specified in full in §6.

7.7 · Invoice / Payment Reference — OPEN (O-1)

This mapping is not settled and is marked open at the team's direction. P7-BE-04 must not be started until it closes.

§18 says only "*Finance/CRM fields*", and there are two candidate homes, with different APIs:

- **Zoho Books** — which **CLAUDE.md** and the stack decision name as the invoicing system ("Sales, invoices, payment status | Zoho").
- **The CRM Invoices module** — which exists in this org, is API-enabled, and is currently unused.

The recommendation on the table is **Books as system of record, CRM Invoices left unused**, because that is what the stack decision already says and splitting invoices across two Zoho products guarantees they disagree. It needs confirming rather than assuming, because it decides which API the worker calls.

One clarification the open decision does **not** need to wait for, because §18 conflates two different things under one row:

	SPONSOR INVOICE	ATHLETE PAYOUT REFERENCE
What it is	BTG bills the sponsor for the campaign	BTG has paid the athlete
SponsorX field	Campaign — needs new fields, schema gap S-6	Earning.reference (exists)
Direction	← from Zoho	Entered by Finance in the SponsorX admin
Why	Zoho issues invoices. SponsorX must never become the invoicing system.	Per gate G-01 , Phase 1 tracks earnings <i>status</i> only; money moves outside the system. There is no Zoho object to sync, and no tax ID is collected or stored (Addendum A6).

Provisional inbound field list, for whichever source wins — all ←, Zoho absolutely SoR, SponsorX storing a reference and a status and nothing more:

ZOHO CONCEPT	SPONSORX TARGET	NOTES
Invoice number	Campaign.invoiceReference (new)	Display only.

ZOHO CONCEPT	SPONSORX TARGET	NOTES
Invoice status	<code>Campaign.paymentStatus</code> (new)	An enum of our own, mapped from Zoho's status set.
Invoice total	(not stored)	<code>Campaign.budget</code> is our number. Storing Zoho's total invites two truths about one campaign.
Balance due	(not stored)	Same reason. If Finance needs it on screen, it is a Books read in the worker with a named retrieval path — never a read-through on a request path.
Payment date	<code>Campaign.paidAt</code> (new)	

8 · Conflict resolution and loop prevention

8.1 · The order of checks

For every inbound Zoho webhook and every outbound push:

- Signature.** Inbound, verify `x-zoho-signature` first. Record the attempt in `WebhookDelivery` whether or not it verifies — §20 requires the log, and an unverified attempt is exactly what you want a record of.
- Echo test.** Hash the payload. Outbound: if `lastSyncOrigin = ZOH0` and `lastSyncHash` is unchanged, drop the write. Inbound: if `lastSyncOrigin = SPONSORX` and the hash matches what we last sent, drop it. This is what stops the ping-pong loop (`P8-INT-04`).
- Concurrent-change test.** If Zoho's `Modified_Time` and our `updatedAt` are both later than the last successful sync, the field changed on both sides.
- Apply the SoR column.** The winner's value is written. **The loser's value is written to `AuditLog` as a `sync.conflict` entry** with both values, the field, and both timestamps. A conflict is never resolved silently — if sponsor names are being fought over, someone needs to be able to see it happened.

Schema gap S-7. Step 3 needs `updatedAt` on every Zoho-touched model. `Athlete` currently has `createdAt` only, and `Sponsor` has neither. Without it there is no way to tell a concurrent change from a stale one, and the sync has to guess.

8.2 · Queueing, retries, ordering

- All Zoho traffic is worker-side, on the Postgres queue (Addendum A3). Zoho credentials live **only** on the worker (`ZOH0_CLIENT_ID` / `SECRET` / `REFRESH_TOKEN`), never on the web app.
- Retries use exponential backoff. Because every write is an upsert on `SponsorX_ID`, a retry is idempotent.
- Ordering is a real constraint.** Account before contact, account before deal, contact before deal. A job whose parent reference is still null re-queues rather than creating a parentless CRM record.
- Zoho's API is rate-limited per org per day. The queue is the throttle; `P8-INT-07`'s backfill must run at a bounded rate and against the sandbox first.

8.3 · Reconciliation

`P8-INT-05` runs nightly: for each object, compare SponsorX rows against Zoho records by `SponsorX_ID` and report — never auto-repair — three categories: in SponsorX but not Zoho (a lost push), in Zoho with a

`SponsorX_ID` we do not recognise (a deleted row or a wrong environment), and present in both with field-level divergence (a missed webhook). Auto-repair on a mapping this new would paper over the bug that caused the divergence.

9 · Never synced

Nothing in this list goes to Zoho in Phase 1, in any object.

CATEGORY	WHY
Passwords, MFA state, session data	Clerk's, not ours. <code>CLAUDE.md</code> .
Bank details	Forbidden outright by §26. Stored nowhere, in any system.
Tax IDs	Not collected in Phase 1 — Addendum A6 / gate G-01. If a task appears to need one, stop and raise it.
Minors' dates of birth, guardian identities and contact details	§4, §11, §26. §6.4.
Fan PII from the QR funnel (<code>RewardEvent.fanEmail</code>)	Consent-gated. <code>P6-INT-01</code> pushes a fan lead only where consent was given, and that is a separate, explicitly-scoped flow with its own counsel-approved wording (<code>P0-LEG-04</code>). No consent, no push — enforced in code, not by convention.
R2 object keys (<code>CreativeAsset.r2Key</code> , <code>RewardToken.qrKey</code>)	Private-bucket paths. Useless outside our signed-URL flow and a needless disclosure of storage layout.
Agreement bodies and <code>bodyHash</code> , <code>AgreementAcceptance</code> rows	The legal record of what text was shown to whom, when, from which IP. It must have exactly one custodian, and that is Postgres. §12.
<code>RewardToken.token</code> , <code>TrackingLink.code</code>	Opaque secrets. A leaked token is a redeemable offer.
Raw metric rows (<code>MetricDaily</code> , <code>LinkEvent</code> , <code>RewardEvent</code>)	Performance data is SponsorX's side of the boundary, per §18's own closing rule. And a number copied into a CRM field loses its provenance label — the credibility failure §22 exists to prevent.
<code>AuditLog</code> , <code>OutboxJob</code> , <code>WebhookDelivery</code>	Internal plumbing.
<code>tenantId</code>	Our multi-tenancy, meaningless to the CRM.

10 · Open decisions

#	DECISION	STATUS	BLOCKS	RECOMMENDATION
O-1	Invoice / payment reference: Zoho Books or the CRM Invoices module?	OPEN — flagged at the team's direction	<code>P7-BE-04</code>	Books as SoR, CRM Invoices unused — consistent with the stack decision. Needs confirming; it selects the API.
O-2	Campaign represented as a Deal , CRM Campaigns module unused.	Recommended, needs sign-off	<code>P8-INT-01</code>	Adopt. Rationale in §4.1: the stock module is an email-marketing object wired to two Zoho extensions.

#	DECISION	STATUS	BLOCKS	RECOMMENDATION
O-3	Breadth of athlete data in the CRM.	Decided for this revision — conservative set (§6.2), expansion path in §6.5	P0-OPS-05	Build the eleven fields. Widen later by decision, never by default.
O-4	Read lead-conversion fields back from Zoho (<code>Converted__s</code> , <code>Converted_Account</code>)?	Proposed	P8-INT-06	Small, read-only, and it closes the inquiry loop — but §18 marks Lead one-way, so it is a scope addition and needs a yes.
O-5	Zoho user ↔ BTG staff mapping, and who owns records the service account creates.	Unresolved	P8-INT-01 , task sync	Add <code>User.zohoUserId</code> (S-5) and nominate a default owner for service-account records. Otherwise CRM records land ownerless and nobody is notified.
O-6	No sandbox org exists; this is production.	Known	P8-INT-07	P0-OPS-04 must deliver one before any backfill runs.

11 · Schema and configuration changes this document requires

Everything the mapping needs that does not exist yet. **Schema items are the handoff to `P3-BE-01`** (the Prisma schema task), whose acceptance already reads *"external IDs + lastSyncOrigin + lastSyncHash on every Zoho-touched model"* — this list is what that sentence resolves to.

#	CHANGE	NEEDED BY
S-1	Add <code>SponsorContact</code> model (§7.2)	§18 row 2, P4-BE-01
S-2	Add <code>Inquiry</code> model (§7.3)	§18 row 3, P8-INT-06
S-3	Add <code>CampaignBrief.zohoDealId</code>	§18 rows 4–5, §4.1
S-4	Add <code>SyncTask</code> model (§7.5)	§18 row 7
S-5	Add <code>User.zohoUserId</code>	Task ownership, O-5
S-6	Add <code>Campaign.invoiceReference</code> , <code>.paymentStatus</code> , <code>.paidAt</code>	§18 row 8 — hold until O-1 closes
S-7	Add <code>updatedAt</code> to every Zoho-touched model (<code>Sponsor</code> , <code>Athlete</code> , <code>Campaign</code> , <code>CampaignBrief</code> , and the new models)	Conflict detection, §8.1 step 3
S-8	Add <code>lastSyncOrigin</code> / <code>lastSyncHash</code> to <code>Campaign</code> and <code>CampaignBrief</code>	They are Zoho-touched but carry no sync markers today
S-9	Decide <code>Property.zohoId</code>	It exists in the schema, but §18 maps no <code>Property</code> object and §6.2 field 10 deliberately sends the property as plain text. Either delete the column or document it as reserved — an unused external-ID column will eventually get populated by someone who assumes it is wired up.

#	CHANGE	NEEDED BY
G-1	Add the SponsorX_ID external field to Accounts , Contacts , Leads , Deals , Tasks (§5.1)	Every sync task. No task currently owns this work — P0-0PS-05 covers only the custom module. Fold it into P0-0PS-05 or raise P0-0PS-06 . Without it, §18's "use external IDs to prevent duplicates" is unimplementable.
G-2	Audit the org's Zoho workflow rules against the SoR columns (§5.3)	Before go-live

12 · Acceptance

Against the task's stated *done when* — "Every object in §18 mapped field-by-field, with direction and system-of-record owner per field":

REQUIREMENT	WHERE	STATUS
All nine §18 objects present	§4 register	✓ Nine rows, each resolved to a concrete Zoho module
Field-by-field mapping	§6.2, §7.1–§7.7	✓ Eight of nine complete. Object 8 is provisional pending O-1 , flagged.
Direction per field	Dir column, every table	✓
System-of-record per field	SoR column, every table	✓ Including two deliberate splits (Deals.Amount , Deals.Stage)
External IDs to prevent duplicates (§18)	§3, §5.1	✓ Specified — and G-1 raised , because the fields do not exist in the org yet
Queued sync with retries (§18)	§8.2	✓
Loop prevention (Addendum A9)	§8.1	✓
SponsorX SoR for network, inventory, orders, deliverables, performance (§18)	§9	✓ None of it is synced
Zoho SoR for the sales relationship and pipeline (§18)	§7.1, §7.4	✓ Stage and contact data are Zoho's, with two named exceptions
Custom-module spec sufficient to build P0-0PS-05	§6	✓ Module, labels, eleven fields, types, picklist values, mandatory flags
Every Zoho field API name verified against the live org	§2	✓ Read from the metadata API on 2026-09-11

Verification performed. Field API names, data types, lengths, mandatory flags and picklist values for **Accounts** , **Contacts** , **Leads** , **Deals** , **Tasks** and **Campaigns** were read from the live **iCARRe Foundation** org via the CRM

metadata API on 2026-09-11. No field name in this document was taken from documentation or memory. **No writes were made to Zoho** — creating the custom module is `P0-0PS-05`, and the external-ID fields are G-1.

13 · Revision history

VERSION	DATE	CHANGE
0.1	2026-09-11	First issue. <code>P0-PM0-08</code> . O-1 left open by decision; conservative athlete field set adopted (O-3).